Learning to Drive a Single Go1 Quadruped to a Target from Any Position: Offline RL and PPO on Real Motion-Capture Data

EKF Physical RL project — `realdata/real_model_v1`

July 19, 2026

Abstract

We learn a control policy that drives a *single* Unitree Go1 quadruped from *any* start position inside a fixed arena to the target $\mathbf{g} = (-2.0, -2.5)\,\mathrm{m}$, subject to a maximum-speed limit and a hard requirement never to leave the map. The policy is grounded in 23 real motion-capture runs. We take great care with the data: each trajectory is truncated at its first goal entry, a goal entry is the *only* event that counts as a terminal state, and each run is classified as reaching / approaching / non-reaching so that the one wandering run that never approaches the goal is never treated as a successful demonstration. We compare three offline off-policy algorithms (TD3+BC, IQL, CQL) trained on the real transitions with PPO trained online in a data-grounded kinematic simulator. Evaluated from 200 random start positions across the whole map, all four reach the goal 100% of the time, but they differ sharply on the constraints: **PPO** reaches with **zero boundary violations** and near-time-optimal paths, whereas the offline methods reach the goal but graze the walls — because the real data never violates the boundary, an offline learner cannot see the penalty that teaches avoidance. We release all policies, animations, and an analysis of the 15,263 onboard camera frames motivating a vision-based extension.

1 Task and World

Single agent. There is one robot; all data are for that robot. **Objective.** From any pose inside the arena, drive to the fixed goal $\mathbf{g} = (-2.0, -2.5)\,\mathrm{m}$ and stop within $\rho = 0.30\,\mathrm{m}$ of it, while (i) respecting a maximum speed and (ii) never leaving the map.

Map. The arena is the rectangle $[-2.15, 2.00] \times [-2.70, 2.40]\,\mathrm{m}$, obtained by expanding the observed mocap extent ($x \in [-1.94, 1.84]$, $y \in [-2.54, 2.26]$) so that it contains the goal, which sits in the bottom-left corner.

Speed limit. From the data, the body linear speed has 99th percentile $\approx 1.4\,\mathrm{m/s}$ and the yaw rate $\approx 2.4\,\mathrm{rad/s}$. We cap the admissible action at $|(v_x, v_y)| \le V_{\max} = 1.5\,\mathrm{m/s}$ and $|\omega| \le W_{\max} = 2.5\,\mathrm{rad/s}$.

2 Careful Data Handling

Each mocap run is resampled to a uniform 10 Hz grid; yaw is taken from the quaternion. We then, per run:

1. classify it by how close it gets to the goal — **REACH** (enters the 0.30 m radius), **approach** (net progress $> 0.5\,\mathrm{m}$ but never enters), or **NON-REACH**;

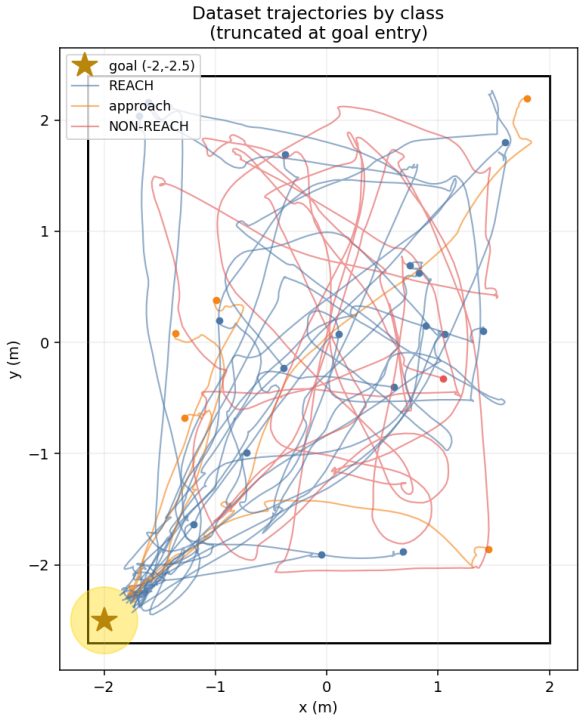


Figure 1: The 23 runs coloured by class (truncated at goal entry). Blue REACH runs converge to the goal; the single red NON-REACH run wanders the arena and never approaches the corner — it must not be treated as a successful demonstration.

2. **truncate at the first goal entry**, discarding post-goal drift so that lingering near the goal is not mislabelled;
3. extract the action $a = (v_x, v_y, \omega)$ as the body-frame twist by finite differencing, then **clip it to the speed limit** (this also removes mocap spikes);
4. set the terminal flag **true only at a genuine goal entry**. A run that merely stops is a *truncation*, not a terminal state; treating it as terminal would wrongly cut the value bootstrap and corrupt learning.

Of the 23 runs, 17 REACH, 5 approach and 1 is NON-REACH (Fig. 1): run `20260624_153602` starts 3.74 m from the goal, wanders for 120 s and *ends farther away* (4.21 m, closest approach 1.28 m). We keep it in the buffer for state-space coverage but, because rewards are progress-based, its transitions receive correctly low values, so value-based offline RL never imitates it. The final dataset has 4,705 transitions with 17 true terminals.

3 MDP and Environment

Observation (position-aware, 9-D). $s = [\,x,\ y,\ \cos\psi,\ \sin\psi,\ g_x^b,\ g_y^b,\ v_x,\ v_y,\ \omega\,]$, where (g_x^b, g_y^b) is the goal in the body frame. The absolute position (x, y) is included so the policy can reason about the walls (goal-relative features alone cannot express where the boundary is).

Action. Body-frame twist $a = (v_x, v_y, \omega)$, clipped to the speed limit inside the environment.

Reward (designed). With $d_t = \|\mathbf{g} - \mathbf{p}_t\|$,

$$r_t = 5\,(d_t - d_{t+1}) \;-\; 0.01\|a_t\|^2 \;-\; 0.02 \;+\; 20 \cdot \mathbb{1}[d_{t+1} < \rho] \;-\; 5 \cdot \mathbb{1}[\text{wall}]. \tag{1}$$

The dense progress term drives goal-reaching; the terminal bonus sharpens the stop; the wall term (active only when the robot pushes against a boundary) teaches map-compliance.

Environment. First-order kinematic body-velocity dynamics — the exact inverse of the action extraction, so data and simulator are consistent. The robot is hard-clamped inside the map (it *cannot* leave), and pushing against a wall incurs the penalty. Episodes reset to **random poses across the whole map**, which is what makes the learned policy work from any position.

4 Algorithms

Three *offline* off-policy methods learn from the real transitions: **TD3+BC**, **IQL** (expectile 0.7, AWR temperature 3.0), **CQL** (SAC backbone, conservative penalty). **PPO** is *on-policy*, so it cannot consume the offline data directly; instead it trains online in the data-grounded environment, where it sees the boundary penalty and random starts. All share 2×256 MLPs, $\gamma = 0.99$ and a best-checkpoint rule that ranks by (success, then fewest wall violations, then final distance).

5 Results

Every method is evaluated from the *same* 200 random start positions across the map (Table 1, Figs. 2–4).

Table 1: Evaluation from 200 random map positions (best checkpoint). Wall-viol. = fraction of episodes with boundary contact away from the cornered goal; path ratio = path length / straight-line distance (1 is optimal).

Method	Success	Wall-viol.	Steps to goal	Path ratio	Type
PPO (winner)	**1.00**	**0.00**	**21**	**0.93**	online in sim
IQL	1.00	0.17	66	1.34	offline
TD3+BC	1.00	0.47	34	1.01	offline
CQL	1.00	0.94	66	2.41	offline

All four reach the goal from any position, so the *go-to-goal* objective is solved by every method. The decisive differences are the *constraints*: PPO never touches a wall and takes essentially time-optimal, $\sim 2\,$s paths at near-maximum speed, while the offline methods — which never observe a boundary violation in the real data — reach the goal but graze the walls (CQL badly so). This is exactly why an *online* learner in the data-grounded simulator is the right tool for the "from any position, stay in the map" requirement, with the offline policies serving as strong data-driven baselines. The winning policy is saved to `policies/best_policy.pt`.

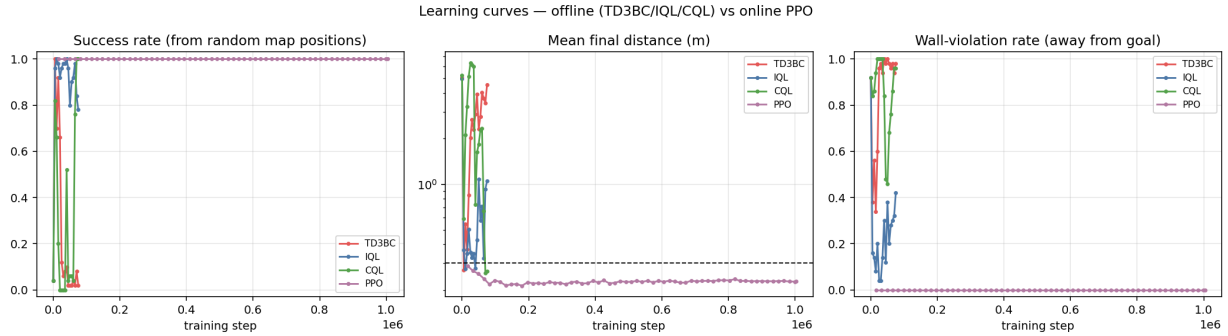
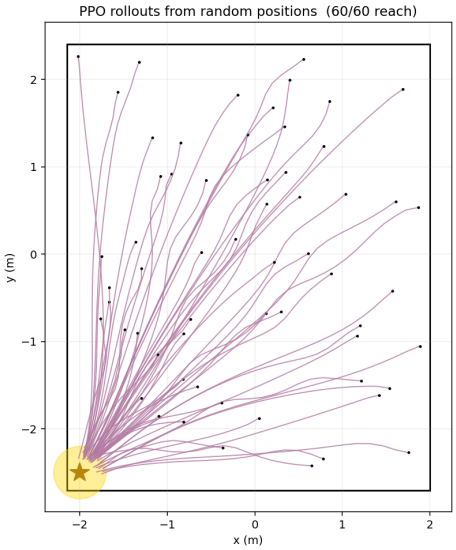
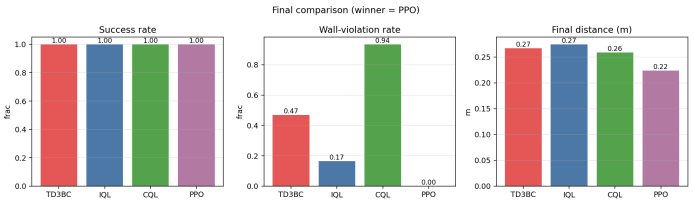


Figure 2: Learning curves (evaluated from random map positions). PPO reaches 100% success with a 0 wall-violation rate; the offline methods reach the goal but retain higher wall-violation rates; TD3+BC and CQL are less stable.



(a) PPO reaches from all random starts, inside the map.



(b) Final metric comparison.

Figure 3: Winner behaviour and comparison. The black rectangle is the map boundary; the gold star/disc is the goal and its success radius.

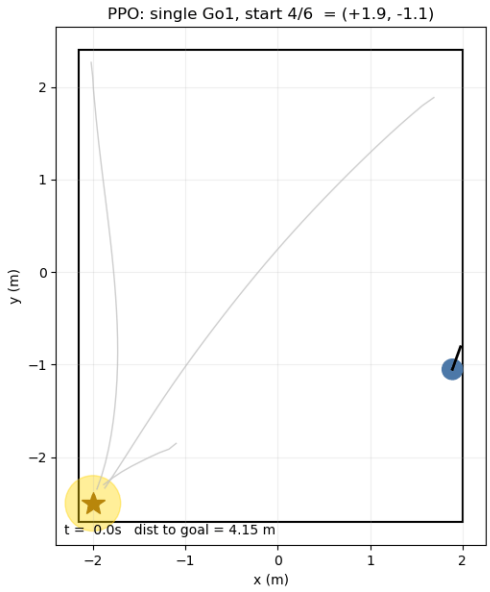


Figure 4: Frame of `animations/single_agent_multistart.gif`: a *single* Go1 (blue, with heading segment) driving to the goal, run from several different start positions *in sequence*. Faded grey curves are the robot's earlier completed runs, illustrating that one policy reaches from anywhere in the map.

6 Onboard Camera Data: a Vision-Based Extension

The policy uses only mocap pose, but the dataset also holds 15,263 unused onboard fisheye frames (480 × 400; mean brightness 71.4, mean Laplacian sharpness 222.8). Simple operators already expose task-relevant structure (Fig. 5): walls and a doorway, stacked cardboard boxes that act as obstacles, and floor markings.

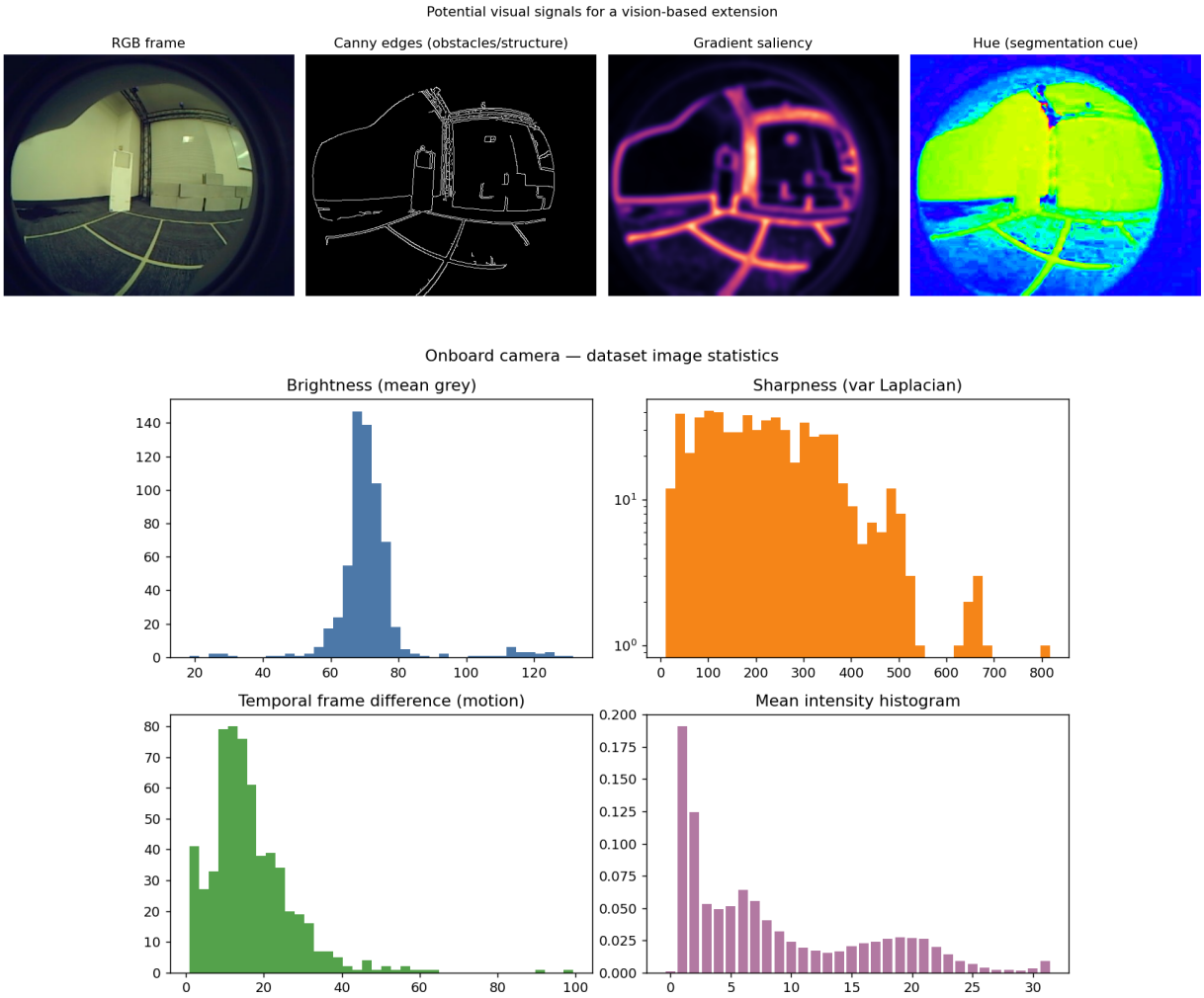


Figure 5: Top: a frame with edge / saliency / hue cues exposing obstacles, free space and structure. Bottom: dataset-wide image statistics.

Proposed extensions. (1) **Vision-conditioned goal-reaching**: replace the mocap position with a small CNN encoder, enabling deployment *without* an external mocap system. (2) **Obstacle-aware navigation**: the visible boxes support a learned collision-avoidance term, complementing the current wall constraint. (3) **Visual-inertial estimation**: fuse the camera with the logged joint/IMU streams (the parent project's "EKF" theme) to recover pose when mocap is unavailable. (4) **Representation pretraining**: the 15k frames support self-supervised temporal-contrastive pretraining (mean inter-frame difference 16.6 confirms substantial motion) to warm-start the encoder. Caveats visible in the data: strong fisheye distortion (calibrate/undistort) and brightness

varying 18–132 across runs (photometric augmentation).

7 Deliverables and Reproducibility

The trained models live in a labeled `models/` folder: one subfolder per model (`ppo_online/`, `iql_offline/`, `td3bc_offline/`, `cql_offline/`), each with its weights, `metrics.json` and a detailed `model_card.md`; the deployed winner is copied to `models/best/`, and `models/MODELS.md` indexes them.

Pipeline: `python -m src.data_pipeline` (build data) → `python -m src.train` (train the four methods, compare, save best) → `python -m src.export_models` (labeled `models/` folder) → `python -m src.visualize` (figures + single-agent animation) → `python -m src.image_analysis`. All runs use the `marl_env` conda env (PyTorch 2.7 + CUDA, RTX 4090) and a fixed seed; hyperparameters are in `src/config.py`.